

Zebra mempool crawler Path Bypasses the Per-Peer Download Slot Limit

Summary

Zebra permits at most 500 concurrent new-transaction downloads. An ordinary attacker can fill all of these slots with only 500 fake transactions, after which transfers from legitimate users are rejected from the mempool. The node does not crash, and CPU and memory usage appear normal, but it can no longer accept new transactions. The primary impact is that transactions remain stuck for a long time or cannot be confirmed.

The latest stable version, zebra v6.3.0, remains affected. Historical versions 6.2.2 and 6.2.3 have the same issue; zcashd is not affected. The issue was reproduced in an isolated Regtest environment.

Mitigating Factors:

- & An attack from a single IP is usually intermittent because the attacker must wait for the crawler to select its connection at random. On average, it can affect the node only for short periods. To approach sustained disruption, the attacker generally needs approximately 15 cooperating IP addresses.
- & The vulnerability only affects the node's ability to receive new transactions. It does not directly steal coins, modify existing blocks, or damage on-disk data. The node can recover on its own after the attack stops and the occupied slots are released.
- & The attack fills a bounded pool of 500 application-defined slots rather than consuming unbounded memory or CPU, so it normally does not cause host OOM or a process crash.
- & The issue exists only in Zebra's crawler processing path. Nodes using zcashd are not affected.

Aggravating Factors:

- & The attack barrier is low: an attacker only needs to establish an ordinary P2P connection like any normal node, with no account, special privilege, or mining power.
- & The most dangerous characteristic is that everything appears normal: the zebra process remains alive while legitimate transactions can no longer enter the mempool.
- & The operational impact is direct: user transfers fail to propagate, merchants and payment gateways do not receive confirmations, exchange deposits and withdrawals stall, and miner block templates may omit fee-paying transactions that would otherwise be available.
- & If multiple Zebra nodes are attacked at the same time, a single-node disruption can expand into broader transaction propagation and confirmation delays.
- & This bypass path remained uncovered after two related official fixes. Simply upgrading to the earlier fixed versions does not eliminate the risk; the latest v6.3.0 still requires a dedicated patch.

Vulnerability Details:

Background: Download-Slot Quotas and the Related CVE

Zebra's mempool download pipeline has two quota layers (zebra/src/components/mempool/downloads.rs):

```
pub const MAX_INBOUND_CONCURRENCY: usize = 500;           // downloads.rs:107
global slots
pub const MAX_INBOUND_CONCURRENCY_PER_PEER: usize = 5;    // downloads.rs:117 per-
peer slots
```

The global limit of 500 prevents download and validation tasks from exhausting memory; a single malicious transaction can consume about 9 MB after deserialization. The per-peer limit of 5 prevents one peer from filling the global pool with fake txids. This was the fix introduced in 4.5.0 for the related vulnerability CVE-2026-52732 (GHSA-4fc2-h7jh-287c, affecting <=4.4.1, CVSS 5.3). The project later added another hardening change, GHSA-m9xx-8rcj-vmgp, to charge actively pushed full transactions, or tx messages, against the per-peer budget. Neither fix covered the crawler path: crawler mempool responses still enter the pipeline with `source=None`, bypassing the per-peer check entirely.

Normal and Abnormal Behavior

Normal case (inv broadcast path, `source=Some`): when a peer proactively advertises a transaction, its source is recorded and the per-peer cap of 5 applies:

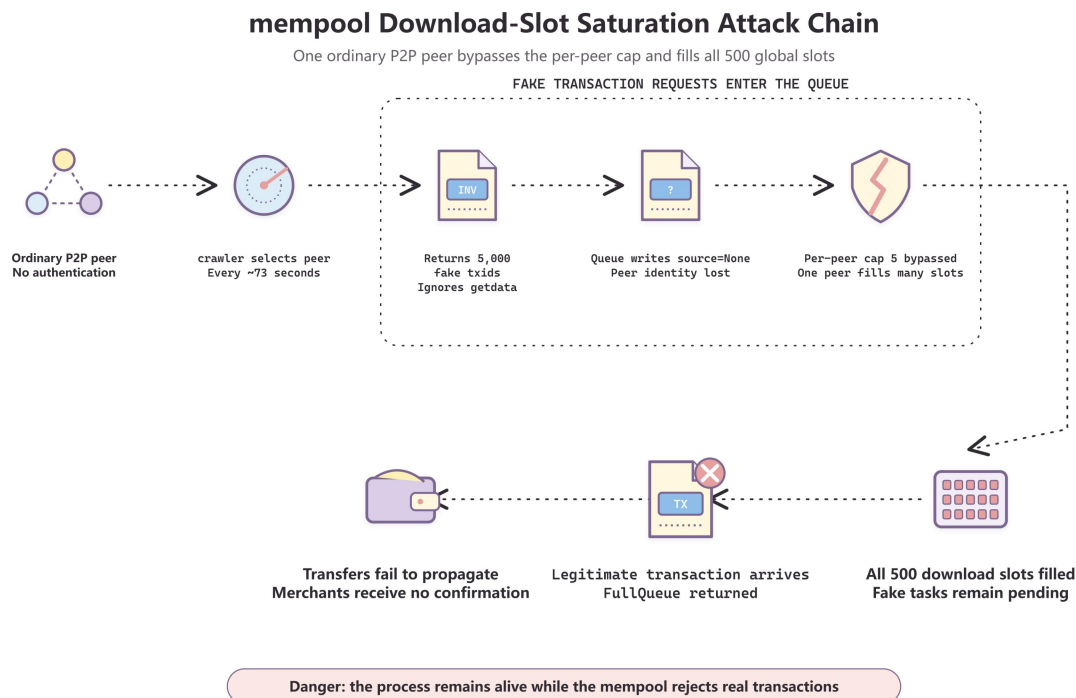
```
inv(txid) -> QueueFromPeer(source=Some(1.2.3.4))
  -> download_if_needed_and_verify(tx, Some(1.2.3.4), _)
  -> if let Some(source) matches -> check pending_per_peer[1.2.3.4] >= 5?
  -> starting with transaction 6, return FullQueue; transactions from other
  peers continue to enter the pool
```

Abnormal case (crawler response path, `source=None`): the node proactively asks a peer for its transaction list, but the returned txids are not associated with any source constraint:

```
crawler sends mempool request -> peer replies with inv(5,000 fake txids)
  -> Request::Queue(txids) (crawler.rs:276)
  -> download_if_needed_and_verify(tx, None, _) (mempool.rs:660)
  -> if let Some(source) is skipped entirely (downloads.rs:364)
  -> all 500 global slots are filled by this one peer
  -> every subsequent new transaction -> FullQueue
```

Technical Details:

Complete Call Chain



Root Cause 1: The Queue Branch Discards the Source Peer (mempool.rs:660)

```
let _result = tx_downloads.download_if_needed_and_verify(tx, None, None);
```

The crawler knows which peer supplied the response because it initiated the connection, but source is hard-coded to None when the download pipeline is called. In contrast, the fixed paths, inv broadcast through QueueFromPeer and tx push, both carry Some(peer_addr). Source information is discarded only on this path.

Root Cause 2: The Per-Peer Cap Is Guarded by if let Some(source) (downloads.rs:364-376)

```
// Per-peer cap: a single advertising peer cannot saturate the queue
// with attacker-supplied fake txids. See `GHSA-4fc2-h7jh-287c`.
if let Some(source) = source {
    let count = self.pending_per_peer.get(&source).copied().unwrap_or(0);
    if count >= MAX_INBOUND_CONCURRENCY_PER_PEER {
        ...
        metrics::counter!("mempool.full_queue.per_peer.total").increment(1);
        return Err(MempoolError::FullQueue);
    }
}
```

The fix for CVE-2026-52732 is correct, but it is enclosed in an Option branch. When source is None, none of the protection code executes. The fix covers paths that pass Some; the crawler passes None, so the fix is bypassed.

Root Cause 3: Explicit Exemption Without Compensating Controls (downloads.rs:323-325)

```
/// When `source` is `Some`, the per-peer cap
/// [`MAX_INBOUND_CONCURRENCY_PER_PEER`] is enforced; crawler-driven and
/// locally-pushed transactions pass `None` and are not capped per peer.
```

The project explicitly documents that the crawler is exempt as a design tradeoff, but provides no compensating control:

- & There is no limit on the number of txids a single crawler response may enqueue; one inv message may contain thousands.
- & A peer that refuses getdata requests is not assigned misbehavior. After fake-txid downloads fail, the source peer is neither penalized nor banned and can inject the same list again in the next cycle.
- & FullQueue is not surfaced as an overload signal on the crawler path. The 4.5.0 fix added overload-based disconnection to the inv path, but not to the crawler path.

Trigger Conditions and Attack Timeline

A real-world attack requires all of the following: the target is a Zebra node with mempool enabled; the attacker can establish an ordinary P2P connection to the target, with no authentication or privilege and one connection per IP; and the crawler selects the attacker's connection to answer a mempool request, choosing three peers at random every 73 seconds. The local reproduction used Regtest with debug_enable_at_height = 0 to force-enable the mempool and debug_force_finished_sync = true to satisfy the synchronization prerequisite. These switches replace only the environmental condition of being synchronized to the tip; the vulnerability itself follows production code paths entirely.

```
T+0s      crawler selects the attacker and sends a mempool request
T+0s      attacker returns inv(5,000 fake txids) -> all 500 slots fill immediately
T+0s+     node issues getdata for each item; attacker never responds
T+20s     TRANSACTION_DOWNLOAD_TIMEOUT expires and slots begin to be released
T+73s     next crawl cycle; the same 5,000 txids are injected again -> all slots
          fill again
...       periodic saturation; the node's mempool remains unavailable for long
          periods
```

Key parameters:

Parameter	Value	Location
Global download slots	500	downloads.rs:107
Per-peer cap (bypassed)	5	downloads.rs:117
Crawl interval	73s	crawler.rs:84
RATE_LIMIT_DELAY		
Peers crawled per cycle	3	crawler.rs:75 FANOUT
Download timeout (slot duration)	~20s (outer cap 73s)	downloads.rs:73,
		downloads.rs:489
Connections per IP	1	
max_connections_per_ip		

Real-World Feasibility

The effect of a single-IP attack is intermittent. With a default peer set of approximately 25 ready connections, the probability of selection in each cycle is $1-(1-1/25)^3$, approximately 11.5%. Each selection saturates the pool for 20 seconds, yielding an expected saturation share of approximately $11.5\% \times 20/73$, or 3%. Because `max_connections_per_ip = 1`, the attacker needs multiple IPs to increase its share:

Attacker IPs	Hit probability per cycle	Approximate time coverage
1	11.5%	~3%
5	~49%	~13%
10	~78%	~21%
15	~94%	~26% (nearly continuous selection)

The theoretical coverage ceiling is approximately $20/73$, or 27%. A real prerequisite was verified by performing batch handshakes with public testnet nodes. Most had full inbound capacity and reset the connection, but some completed and maintained the handshake, confirming that becoming a peer without being banned is feasible. For an honest severity assessment, this is not a one-click permanent outage: the score must reflect AC:H because selection is probabilistic and sustained impact requires multiple cooperating IPs; it must not be scored as AC:L.

Dynamic Evidence

Latest-version test environment: zebra v6.3.0 in fully offline Regtest. The SATURATED verdict required all three conditions: 1) `mempool_currently_queued_transactions = 500` in metrics, proving the global pool was full; 2) `mempool_full_queue_per_peer_total` was absent from the complete metrics output, proving the per-peer cap never triggered; and 3) `fake_peer.log` contained "<- mempool #N: returned 5000 fake txids", aligning the injection event with saturation in time.

```

queued=500.0 (cap=500); per_peer present=False (expect False);
MempoolTransactionIds=1.0
VERDICT: F-04 reproduced successfully -- one fake peer filled all 500 global
download slots through the crawler path,
        and the per-peer cap=5 never triggered (source=None exemption,
downloads.rs:364)

```

Control logic: if the per-peer cap applied to the crawler, one peer could occupy at most 5 slots. The observed value was 500, a 100x difference. A supporting characteristic of the issue is that during saturation zebrad used only 0.1-0.2% CPU, memory remained stable at about 60 MB, and the process did not crash. The consumed resource was a bounded pool of 500 logical quotas, not physical resources.

Historical Version Verification (Local Source Comparison: 6.2.2 and 6.2.3)

Key code	6.2.2 (original report)	6.2.3 (this verification)
Global slots 500	downloads.rs:106	downloads.rs:107 PASS
Per-peer cap 5	downloads.rs:116	downloads.rs:117 PASS
None exemption comment	downloads.rs:318-320	downloads.rs:323-325
PASS		
if let Some(source) guard	downloads.rs:359	downloads.rs:364 PASS
Queue branch passes None	mempool.rs:619	mempool.rs:660 PASS
crawler Queue call	crawler.rs:276	crawler.rs:276 PASS

Conclusion: versions 6.2.2, 6.2.3, and the latest stable version v6.3.0 are affected. The crawler-path exemption remains after both the GHSA-4fc2 and GHSA-m9xx fixes.

Difference from CVE-2026-52732

Dimension	CVE-2026-52732	This vulnerability
Affected versions	<=4.4.1 (fixed in 4.5.0) (latest stable still affected)	6.2.2 / 6.2.3 / 6.3.0
Root cause	No per-peer accounting at all	Accounting exists, but
crawler passes	None and the check is skipped	
Trigger path	Proactive inv broadcast (source=Some)	crawler mempool response (source=None)
Global slots	25 at the time	500
Per-peer cap	None	5, ineffective for crawler

Security Impact

- & A single ordinary peer can cause intermittent denial of new-transaction admission in the target mempool. Cooperation among multiple IPs, approximately 15, can approach sustained disruption of one node.
- & Affected parties include wallet users whose transfers appear to vanish, merchants and payment gateways that do not receive confirmations, exchange hot wallets with delayed deposits and withdrawals, and miners whose block templates omit fee-paying transactions.
- & Attacking multiple nodes at once can expand the impact into network-wide confirmation delays.
- & The impact is limited to availability: it does not touch funds, consensus, or on-disk state, and service recovers after the attack stops.

Long-Term Remediation Recommendations

- & Propagate the source peer through the crawler path. The crawler knows which connection supplied the response; include that address in Request::Queue so download_if_needed_and_verify receives Some(peer) and the per-peer cap of 5 applies naturally, matching the GHSA-m9xx fix for the tx push path.

- & Apply a separate total limit to crawler responses. Give each crawl cycle an independent txid enqueue budget far below 500 and decouple it from the global cap.
- & Add misbehavior attribution. Repeated download failures for txids returned in a crawler response should assign misbehavior points to the source peer and ban it at the threshold. downloads.rs already has a misbehavior_sender pipeline that can be connected for this purpose.
- & Surface FullQueue as an overload signal. The crawler path should also disconnect peers that continuously inject fake txids, aligning it with the 4.5.0 overload mechanism.
- & Reevaluate the None exemption. Remove or narrow the exemption codified by the design comment at downloads.rs:323-325; it is the root cause of this vulnerability.

Recommended regression tests:

- & Test 1: On the inv broadcast path with source=Some, transaction 6 and later receive FullQueue -- existing test; retain for regression protection.
- & Test 2: Inject more than 500 fake txids through a crawler response and verify that one peer occupies no more than MAX_INBOUND_CONCURRENCY_PER_PEER slots -- new test for this vulnerability.
- & Test 3: After a crawler response, verify that the per_peer counter increments and matches the source -- new test.
- & Test 4: Verify that a peer refusing getdata is assigned misbehavior and eventually banned -- new test, paired with remediation 3.
- & Test 5: After slot saturation recovers, verify that legitimate transactions can enter the pool normally -- new test.